

Django Module 4 Important

1. Outline a step to activate the Django admin site and create a superuser.

Steps to Activate Django Admin Site and Create a Superuser

Step 1: Ensure Admin App is Installed

Django provides a built-in admin application. Verify that 'django.contrib.admin' is present in `INSTALLED_APPS` inside `settings.py`.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
]
```

Step 2: Configure Admin URL

Map the admin URL in the project's `urls.py` file.

```
from django.contrib import admin  
from django.urls import path  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
]
```

This allows Django to serve the admin interface at the `/admin/` URL.

Step 3: Apply Migrations

Run the migration command to create the database tables required by Django's authentication and admin system.

```
python manage.py migrate
```

Step 4: Create a Superuser

Create an administrator account using:

```
python manage.py createsuperuser
```

Enter the required details:

Username

Email Address

Password

The superuser has full access to the Django admin panel.

Step 5: Start the Development Server

Run the Django server:

```
python manage.py runserver
```

Step 6: Open the Admin Site

Open a browser and visit:

```
http://127.0.0.1:8000/admin/
```

Step 7: Login with Superuser Credentials

Enter the username and password created in Step 4.

After successful login, the Django Admin Dashboard is displayed, allowing you to manage users, models, and database records through a web interface.

2. Explain the use of `list_display`, `search_fields`, and `readonly_fields` in Django admin.

`list_display`, `search_fields`, and `readonly_fields` in Django Admin

Django Admin provides options to customize how models are displayed and managed in the admin panel.

1. `list_display`

`list_display` is used to specify which model fields should be displayed in the admin list view.

Example

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['student_id', 'name', 'email', 'department']
```

Purpose

- Displays selected fields as columns in the admin panel.
 - Makes records easier to view and manage.
 - Improves readability of the admin interface.
-

2. `search_fields`

`search_fields` enables a search box in the admin panel to search records based on specified fields.

Example

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    search_fields = ['name', 'email']
```

Purpose

- Allows administrators to search records quickly.
 - Supports searching on text-based fields.
 - Useful when the database contains many records.
-

3. `readonly_fields`

`readonly_fields` makes specified fields non-editable in the admin panel.

Example

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    readonly_fields = ['student_id']
```

Purpose

- Prevents modification of important fields.
- Useful for auto-generated or system-managed values.
- Ensures data integrity.

Combined Example

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):

    list_display = ['student_id', 'name', 'email', 'department']

    search_fields = ['name', 'email']

    readonly_fields = ['student_id']
```

Option	Purpose
<code>list_display</code>	Displays selected fields as columns
<code>search_fields</code>	Adds search functionality
<code>readonly_fields</code>	Makes fields non-editable in admin panel

3. Demonstrate cookies and session in Django.

Cookies and Sessions in Django

Cookies and Sessions are used to store user-related information between requests.

Cookies

A cookie is a small piece of data stored on the client's browser. It is sent back to the server with every request.

Setting a Cookie

```
from django.http import HttpResponse

def set_cookie(request):
    response = HttpResponse("Cookie Set")
    response.set_cookie('username', 'Arun')
    return response
```

Getting a Cookie

```
from django.http import HttpResponse
```

```
def get_cookie(request):
    username = request.COOKIES.get('username')
    return HttpResponse(username)
```

Deleting a Cookie

```
from django.http import HttpResponse
```

```
def delete_cookie(request):
    response = HttpResponse("Cookie Deleted")
    response.delete_cookie('username')
    return response
```

Purpose of Cookies

- Store small amounts of user data.
 - Remember user preferences.
 - Maintain user information between requests.
 - Stored on the client side.
-

Sessions

A session stores user data on the server side. Django creates a unique session ID and stores it in a cookie.

Setting Session Data

```
from django.http import HttpResponse
```

```
def set_session(request):
    request.session['username'] = 'Arun'
    return HttpResponse("Session Set")
```

Getting Session Data

```
from django.http import HttpResponse
```

```
def get_session(request):
    username = request.session.get('username')
    return HttpResponse(username)
```

Deleting Session Data

```
from django.http import HttpResponse
```

```
def delete_session(request):
    del request.session['username']
    return HttpResponse("Session Deleted")
```

Destroying Entire Session

```
from django.http import HttpResponse
```

```
def clear_session(request):
    request.session.flush()
    return HttpResponse("Session Cleared")
```

Purpose of Sessions

- Store user-specific data securely.
- Maintain login information.
- Store shopping cart data.
- Stored on the server side.
- More secure than cookies.

Difference Between Cookies and Sessions

Cookies	Sessions
Stored on client browser	Stored on server
Less secure	More secure
Limited storage	Larger storage
User can view or modify data	User cannot directly access data
Uses <code>set_cookie()</code>	Uses <code>request.session</code>

Example Uses:

- Cookies → Remember username, language preference.
- Sessions → Login information, shopping cart, user authentication.

4. Explain how to handles file upload in Django with an example.

Handling File Upload in Django

Django provides `FileField` and `request.FILES` to upload files from users and store them on the server.

Step 1: Create Model

In `models.py`:

```
from django.db import models

class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to='documents/')
```

- `FileField` is used to upload files.
 - `upload_to='documents/'` stores uploaded files inside the `documents` folder.
-

Step 2: Create Form

In `forms.py`:

```
from django import forms
from .models import Document

class DocumentForm(forms.ModelForm):

    class Meta:
        model = Document
        fields = ['title', 'file']
```

Step 3: Create View

In `views.py`:

```
from django.shortcuts import render
from .forms import DocumentForm

def upload_file(request):

    if request.method == 'POST':
        form = DocumentForm(
            request.POST,
            request.FILES
        )

        if form.is_valid():
```

```

        form.save()

    else:
        form = DocumentForm()

    return render(
        request,
        'upload.html',
        {'form': form}
    )

```

- `request.FILES` contains uploaded files.
- `form.save()` stores the file and its details.

Step 4: Create Template

`upload.html`

```

<form method="POST"
    enctype="multipart/form-data">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Upload
    </button>

</form>

```

Important:

`enctype="multipart/form-data"`

is required for file uploads.

Step 5: Configure Media Settings

In `settings.py`:


```
MEDIA_URL = '/media/'
```

```
MEDIA_ROOT = BASE_DIR / 'media'
```

Step 6: Configure URLs

In `urls.py`:

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('upload/',
        views.upload_file),
]
```

Working

1. User selects a file in the form.
2. Form sends data using POST request.
3. Uploaded file is available in `request.FILES`.
4. Django saves the file in the location specified by `upload_to`.
5. File information is stored in the database.

Advantages

- Easy file handling using `FileField`.
- Supports documents, PDFs, images, and other file types.
- Files are stored separately from database records.
- Integrates with Django forms and models.

5. Compare FBVs and CBVs with an example for CRUD operations.

Comparison of FBVs and CBVs for CRUD Operations

In Django, CRUD (Create, Read, Update, Delete) operations can be implemented using:

- Function-Based Views (FBVs)
- Class-Based Views (CBVs)

Difference Between FBVs and CBVs

FBVs	CBVs
Written using functions	Written using classes
Simple and easy to understand	More reusable and organized
Suitable for small applications	Suitable for large applications
More code for CRUD operations	Less code using generic views
Logic is written inside functions	Logic is handled through class methods

CRUD Using Function-Based Views (FBVs)

Create

```
from django.shortcuts import render, redirect
from .forms import StudentForm

def student_create(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect('/')

    else:
        form = StudentForm()

    return render(request, 'form.html', {'form': form})
```

Read

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.all()

    return render(
        request,
        'list.html',
        {'students': students}
    )
```

Update

```
from django.shortcuts import render, redirect
from .models import Student
from .forms import StudentForm
```

```
def student_update(request, id):

    student = Student.objects.get(id=id)

    form = StudentForm(
        request.POST or None,
        instance=student
    )

    if form.is_valid():
        form.save()
        return redirect('/')

    return render(
        request,
        'form.html',
        {'form': form}
    )
```

Delete

```
from django.shortcuts import redirect
from .models import Student
```

```
def student_delete(request, id):

    student = Student.objects.get(id=id)

    student.delete()

    return redirect('/')
```

CRUD Using Class-Based Views (CBVs)

Create

```
from django.views.generic import CreateView
from .models import Student
```

```
class StudentCreateView(CreateView):
    model = Student
```

```
fields = '__all__'  
success_url = '/'
```

Read

```
from django.views.generic import ListView  
from .models import Student
```

```
class StudentListView(ListView):  
    model = Student
```

Update

```
from django.views.generic import UpdateView  
from .models import Student
```

```
class StudentUpdateView(UpdateView):  
    model = Student  
    fields = '__all__'  
    success_url = '/'
```

Delete

```
from django.views.generic import DeleteView  
from .models import Student
```

```
class StudentDeleteView(DeleteView):  
    model = Student  
    success_url = '/'
```

Advantages of FBVs

- Easy to learn.
- Explicit control over logic.
- Suitable for simple applications.

Advantages of CBVs

- Reduces code duplication.
- Provides reusable generic views.
- Better suited for large projects.
- Faster CRUD development using built-in classes.

Common CBVs for CRUD:

Operation	CBV
Create	CreateView
Read	ListView, DetailView
Update	UpdateView
Delete	DeleteView

6. Explain overriding the default user model and give one example of adding a new field.

Overriding the Default User Model in Django

Django provides a built-in `User` model for authentication. Sometimes applications require additional user information that is not available in the default model. In such cases, the default user model can be overridden by creating a custom user model.

Steps to Override the Default User Model

Step 1: Create a Custom User Model

In `models.py`:

```
from django.contrib.auth.models import AbstractUser
from django.db import models
```

```
class CustomUser(AbstractUser):
    phone_number = models.CharField(max_length=15)
```

Here:

- `AbstractUser` provides all fields of Django's default user model.
- `phone_number` is a new custom field added to the user model.

Step 2: Configure `settings.py`

```
AUTH_USER_MODEL = 'myapp.CustomUser'
```

This tells Django to use the custom user model instead of the default one.

Step 3: Create and Apply Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Example: Adding a New Field

```
from django.contrib.auth.models import AbstractUser
from django.db import models
```

```
class CustomUser(AbstractUser):
    phone_number = models.CharField(max_length=15)
```

Fields Available

```
username
first_name
last_name
email
password
phone_number
```

Purpose

- Extend the default user model.
- Store additional user information.
- Customize authentication-related data.
- Avoid creating separate profile models for simple requirements.

Advantages

- Supports application-specific user data.
- Keeps user-related information in one model.
- Integrates with Django authentication system.
- Easy to add custom fields such as phone number, address, or date of birth.

7. Explain UpdateView and DeleteView? Explain with one example.

UpdateView

UpdateView is a generic Class-Based View (CBV) used to update existing records in the database. It automatically displays a form with existing data and saves the modified data.

Example

```
from django.views.generic import UpdateView
from .models import Student
```

```
class StudentUpdateView(UpdateView):
    model = Student
    fields = ['name', 'email', 'department']
    template_name = 'student_form.html'
    success_url = '/'
```

URL Mapping

```
from django.urls import path
from .views import StudentUpdateView
```

```
urlpatterns = [
    path('update/<int:pk>/',
        StudentUpdateView.as_view(),
        name='student_update'),
]
```

Purpose:

- Retrieves an existing record.
- Displays data in a form.
- Updates the record after submission.
- Reduces code compared to FBVs.

DeleteView

DeleteView is a generic Class-Based View (CBV) used to delete records from the database.

Example

```
from django.views.generic import DeleteView
from .models import Student
```

```
class StudentDeleteView(DeleteView):
    model = Student
    template_name = 'student_confirm_delete.html'
    success_url = '/'
```

URL Mapping

```
from django.urls import path
from .views import StudentDeleteView
```

```
urlpatterns = [
    path('delete/<int:pk>/',
        StudentDeleteView.as_view(),
        name='student_delete'),
]
```

Purpose:

- Retrieves the selected record.
- Displays a confirmation page.
- Deletes the record after confirmation.
- Redirects to [success_url](#).

Difference Between UpdateView and DeleteView

UpdateView	DeleteView
Updates existing records	Deletes existing records
Displays editable form	Displays confirmation page
Saves modified data	Removes data permanently
Uses UpdateView	Uses DeleteView

CRUD Mapping

- Create → [CreateView](#)
- Read → [ListView](#), [DetailView](#)
- Update → [UpdateView](#)
- Delete → [DeleteView](#)

8. Explain groups and decorators used in Django?

Groups in Django

Groups are used to organize users and assign permissions to multiple users at once. Instead of assigning permissions individually, permissions can be assigned to a group, and all users in that group inherit those permissions.

Example

```
from django.contrib.auth.models import Group

group = Group.objects.create(name='Managers')
```

Adding a User to a Group

```
from django.contrib.auth.models import User, Group

user = User.objects.get(username='arun')
group = Group.objects.get(name='Managers')

user.groups.add(group)
```

Purpose

- Manage user permissions efficiently.
 - Assign the same permissions to multiple users.
 - Simplify access control.
-

Decorators in Django

Decorators are functions used to modify the behavior of views. They are placed above a view function using the `@` symbol.

1. `login_required`

Used to allow access only to authenticated users.

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
```

```
@login_required
def dashboard(request):
    return HttpResponse("Welcome User")
```

Purpose:

- Restricts access to logged-in users.
 - Redirects anonymous users to the login page.
-

2. `permission_required`

Used to check whether a user has a specific permission.

```
from django.contrib.auth.decorators import permission_required
```

```
@permission_required('myapp.add_student')
def add_student(request):
    pass
```

Purpose:

- Grants access only to users with required permissions.
-

3. Custom Decorator Example

```
from django.http import HttpResponse

def admin_only(view_func):

    def wrapper(request, *args, **kwargs):

        if request.user.is_staff:
            return view_func(request, *args, **kwargs)

        return HttpResponse("Access Denied")

    return wrapper
```

Usage:

```
@admin_only
def admin_page(request):
    return HttpResponse("Admin Page")
```

Difference Between Groups and Decorators

Groups	Decorators
Used for managing permissions	Used for controlling view access
Applied to users	Applied to views
Organizes users into roles	Adds restrictions or behavior to views
Example: Admin, Manager, Student	Example: @login_required

Advantages

- **Groups:** Simplify permission management.
- **Decorators:** Improve security and access control.
- Both help implement authentication and authorization in Django applications.